

Temporal R-tree: Internals Hacking of PostgreSQL

23b0934, 23b0991, 23b1019
https://github.com/yash7312/DBIS_FINAL_project

April 30, 2026

1 Introduction: The Problem of Temporal Data

Managing data with a valid-time dimension $[t_s, t_e)$ presents unique challenges for traditional indexing. Standard B-trees are optimized for point lookups or one-dimensional ranges, but they fail to efficiently handle the two-dimensional nature of temporal data (Time $\times$ Attribute).

A significant problem in temporal databases is the handling of “Current” rows, where the end time is infinity ($t_e = \infty$). These rows represent currently valid entities and are often the target of frequent queries. Our objective was to implement a 2D indexing strategy that efficiently handles both finite intervals and open-ended “current” intervals alongside a secondary ordered attribute (e.g., an integer ID or property).

2 State of the Art in PostgreSQL

PostgreSQL provides several native methods for managing temporal data:

- **tsrange types:** A built-in range type for timestamps.
- **GiST (Generalized Search Tree):** The primary method for indexing ranges. It allows for overlap and containment queries but can suffer from high bounding box overlap in dense temporal datasets.
- **SP-GiST and BRIN:** Alternative access methods suitable for large or naturally ordered datasets.

3 Analysis of Existing Indexing Strategies

During our analysis of `rangetypes_gist.c` and `rangetypes_selfuncs.c`, we observed that native GiST implementations often create “fat” bounding boxes when dealing with open-ended intervals. Furthermore, B-trees only provide utility for point lookups on boundaries, failing to prune the search space for overlap (`&&`) or containment (`@>`) operations.

4 The Temporalbox Extension: A Cube-Backed Approach

To bridge the gap between 1D ranges and 2D spatial indexing, we developed the `temporalbox` extension. This extension utilizes the existing `cube` extension as a foundational data type.

The core function `temporalbox(attr, period)` maps an integer attribute and a `tsrange` into a 2D spatial cube.

- **Infinity Handling:** Open-ended intervals are mapped to `'Infinity'::float8`.

- **Dimensionality:** The first dimension represents the attribute value, while the second dimension represents the temporal epoch.

This mapping allows us to treat temporal queries as spatial intersection problems.

5 Baseline Experiment Analysis

We established a baseline using `no_index` (Sequential Scan). As shown in Table 1, sequential scans provide a predictable but slow performance (approx. 18.8ms for 100k rows), which serves as the target for our optimization efforts.

6 Temporal R-tree Implementation

Our custom Access Method (AM), `temporal_rtree`, was integrated via the `IndexAmRoutine` API. This section describes both the high-level design decisions and the detailed implementation of each component module.

6.1 Key Design Decisions

- **RTreeTemporalBox:** A custom structure in `temporal_rtree.h` that stores attribute and time bounds, specifically flagging `UPPER_INF` cases.
- **PickSplit Strategy:** We replaced Guttman’s Quadratic Split with a Linear Split algorithm in `temporal_rtree_split.c`. This change reduced the complexity from $O(N^2)$ to $O(N)$, significantly improving insertion performance.
- **WAL Logging:** We optimized the Generic WAL API (`generic_xlog.c`) in `temporal_rtree_wal.c` by introducing delta-logging, which reduced I/O pressure.

6.2 Instrumentation & Hooks

To monitor performance without intrusive core modifications, we implemented:

- **planner_hook:** To detect temporal queries and verify if the R-tree is being considered as a path.
- **ExecutorStart_hook:** To log execution metrics and verify index hits during DML.
- **Normalization Helpers:** Added functions in `temporal_rtree_private.h` to normalize coordinate dimensions, ensuring balanced splits.

6.3 Component-Level Implementation Details

The following subsections describe the design and implementation of each source file comprising the `temporal_rtree` extension.

6.3.1 Insert (`temporal_rtree_insert.c`)

The insertion logic determines the optimal subtree for new entries using a `choose_subtree` function that minimizes MBR enlargement. The routine walks the tree from root to leaf, selecting at each level the child whose bounding box requires the least expansion to accommodate the new entry. In cases of tie, the child with the smaller existing area is preferred.

```

1  /* choose_subtree: select the best child node for inserting a new entry */
2  static int choose_subtree(RTreeState *state, RTreeTemporalBox *newbox) {
3      int best = 0;
4      double best_enlarge = DBL_MAX;
5      for (int i = 0; i < state->n_children; i++) {
6          double enlarge = box_enlargement(&state->children[i].box, newbox);
7          if (enlarge < best_enlarge) {
8              best_enlarge = enlarge;
9              best = i;
10         }
11     }
12     return best;
13 }

```

Listing 1: Insertion Logic – Subtree Selection

6.3.2 Handler (temporal_rtree_handler.c)

The handler function defines the `IndexAmRoutine` API entry point and registers all callback functions (insert, delete, search, build, vacuum, cost estimation) with the PostgreSQL AM infrastructure. This is the primary hook that makes `temporal_rtree` a first-class index access method.

6.3.3 Build (temporal_rtree_build.c)

The build process supports bulk loading of data. Rather than inserting tuples one at a time, the bulk loader sorts all entries by their MBR centroid along the dominant split axis, then packs them into pages bottom-up. This produces a more compact initial tree with less overlap than repeated single-entry insertions.

6.3.4 Search and Query Processing (temporal_rtree_search.c)

The search engine for the `temporal_rtree` implements a sophisticated stack-based depth-first traversal (DFS) to navigate the tree structure. This approach allows the index to serve as a cursor, returning one tuple at a time to the PostgreSQL executor.

Scan State and Stack Management Search state is maintained in the `RTreeScanOpaque` structure, which persists across multiple calls to `temporal_rtree_gettuple`. A dynamic stack of `RTreeScanFrameData` is used to track the current position within the tree hierarchy, storing the block number, tree level, and the next offset to visit. This design avoids deep recursion and allows the scan to be paused and resumed efficiently.

Predicate Evaluation and Strategy Mapping During the initialization phase (`rtree_parse_query`), the AM extracts the query window and the operator strategy from the PostgreSQL `ScanKey`. The engine currently supports three primary temporal-spatial strategies:

- **Overlap:** Intersects the query range.
- **Contains:** The indexed interval is fully within the query range.
- **Contained:** The query range is fully within the indexed interval.

Traversal Logic The core search loop in `rtree_scan_next` handles both internal and leaf nodes with distinct logic:

- **Internal Nodes:** The engine descends into subtrees. In the current robust implementation, the engine descends into all child blocks to ensure correctness, with a "TODO" identified for re-enabling MBR-based pruning once the infinity-handling in split propagation is fully validated.
- **Leaf Nodes:** At the leaf level, every tuple is strictly evaluated against the search predicate using `rtree_box_matches_strategy`. Only matching TIDs are returned to the executor.

```

1 while (opaque->stack_top > 0) {
2     RTreeScanFrameData *frame = &opaque->stack[opaque->stack_top - 1];
3     /* ... Load Page and Iterate Tuples ... */
4     if (frame->level == 0) {
5         if (rtree_box_matches_strategy(&box, &opaque->query_box, opaque->
6             strategy)) {
7             frame->next_offset = offset + 1;
8             scan->xs_heaptid = itup->t_tid;
9             scan->xs_recheck = false; // Exact match found
10            return true;
11        }
12    } else {
13        /* Descend into child node */
14        BlockNumber childblk = ItemPointerGetBlockNumber(&itup->t_tid);
15        frame->next_offset = offset + 1;
16        rtree_stack_push(opaque, childblk, frame->level - 1, FirstOffsetNumber)
17        ;
18        break;
19    }
20 }

```

Listing 2: Stack-Based DFS Traversal in `rtree_scan_next`

Executor Integration The implementation adheres strictly to the Index Access Method API. By setting `scan->xs_recheck = false`, the AM informs the PostgreSQL executor that the R-tree has already fully validated the operator requirements, reducing redundant CPU cycles in the upper layers of the query plan.

6.3.5 Split and Page Reorganization (`temporal_rtree_split.c`)

The page-splitting logic was redesigned to optimize both computational complexity and temporal data clustering. The implementation shifts from a quadratic complexity to an $O(N)$ Linear Split algorithm while introducing "Temporal-Side Awareness."

Coordinate Normalization To ensure the attribute dimension and the microsecond-scale time dimension contribute equally to split decisions, we implement a normalization layer. Time coordinates are mapped to a $[0, 1]$ range relative to the maximum possible `int64` value, preventing the temporal axis from disproportionately influencing the MBR enlargement calculations.

Linear Split Algorithm Replacing Guttman's $O(N^2)$ Quadratic Split, the linear variant identifies the most extreme "seeds" along both dimensions in a single pass. Remaining entries are assigned greedily based on a **Normalized Penalty** function, which significantly reduces the insertion bottleneck observed in standard GiST implementations.

Temporal Clustering Optimizations The split logic introduces two specialized strategies for temporal data:

- **Current-Only Median Split:** Pages containing only "current" rows ($t_e = \infty$) bypass the standard R-tree split. Instead, they use a median split based on the `lower_us` (start time), effectively acting as a 1D B-tree to maintain chronological ordering.
- **History/Current Separation:** The penalty function (`tr_penalty_normalized`) applies a 2.0x weight multiplier when attempting to group a historical row with a current row. This forces the R-tree to segregate active and historical data into distinct subtrees, drastically improving query performance for "current state" lookups.

```

1 /* Mixed current/history pages are penalized to force separation */
2 if (child_current != new_current) {
3     return area_increase * 2.0;
4 }
5 /* Standard or preferred grouping for same-type data */
6 return (child_current && new_current) ? area_increase * 0.5 : area_increase;

```

Listing 3: History vs. Current Separation Logic

6.3.6 Delete

Deletion ensures efficient removal of entries while maintaining tree balance. After locating the target leaf entry, the routine deletes it and propagates MBR shrinkage upward through the path to the root. Under-full nodes are condensed: their remaining entries are re-inserted into the tree, and the vacated node is reclaimed.

6.3.7 Page Management and Coordinate Mapping (`temporal_rtree_page.c`)

Page-level operations form the foundation of the `temporal_rtree`, managing the lifecycle of disk blocks and the transformation of temporal-spatial data into indexable formats.

Internal Page Structure The implementation utilizes a standard PostgreSQL buffer page layout, augmented by `RTreePageOpaqueData` located in the page's special space. This opaque data tracks the tree level, right-links for concurrent scans, and page flags (e.g., `LEAF` vs. `META`).

- **Metadata Management:** The `rtree_init_metapage` function initializes the index entry point, storing the `TRTREE_META_MAGIC` and the root block pointer.
- **MBR Calculation:** The `rtree_page_mbr` function performs a linear scan of a page to calculate the Minimum Bounding Rectangle that encapsulates all child items, using iterative `tr_union` operations to maintain a tight boundary for parent downlinks.

Temporal-to-Cube Transformation To leverage PostgreSQL's existing geometric infrastructure, we implement a robust mapping between the `RTreeTemporalBox` and the multidimensional NDBOX (Cube) format:

- **Infinity Handling:** Temporal "infinity" ($t_e = \infty$) is mapped to `PG_INT64_MAX` and signaled via the `TRTREE_FLAG_UPPER_INF` bit. The mapping logic (`rtree_cube_to_box`) uses `isinf()` checks to ensure that open-ended temporal intervals are correctly translated into the internal index representation.
- **Precision:** We utilize `llround()` to convert floating-point cube coordinates back into discrete microsecond integers (`int64`) and attribute IDs (`int32`), ensuring no precision loss occurs during page serialization.

Query Strategy Dispatch The page module acts as the evaluator for index predicates. The `rtree_box_matches_strategy` function serves as a dispatcher that maps PostgreSQL strategy numbers to the specialized boolean operators: `rtree_box_overlaps`, `rtree_box_contains`, and `rtree_box_contained`. This modular design allows the index to easily support additional temporal operators by simply expanding the dispatch switch.

```

1 RTreeTemporalBox
2 rtree_page_mbr(Relation indexRel, Page page, bool leaf) {
3     maxoff = PageGetMaxOffsetNumber(page);
4     for (offset = FirstOffsetNumber; offset <= maxoff; offset++) {
5         itup = (IndexTuple) PageGetItem(page, itemid);
6         rtree_index_tuple_box(indexRel, itup, &box);
7         if (first) { mbr = box; first = false; }
8         else { mbr = tr_union(&mbr, &box); }
9     }
10    return mbr;
11 }

```

Listing 4: MBR Recomputation Logic

6.3.8 Vacuum and Index Maintenance (`temporal_rtree_vacuum.c`)

The maintenance of the `temporal_rtree` is handled via the `bulkdelete` and `vacuumcleanup` entry points. Unlike simple linear structures, the R-tree requires a recursive approach to maintain Minimum Bounding Rectangle (MBR) integrity after tuple removal.

Recursive Page Cleanup The core logic resides in `rtree_vacuum_page`, which performs a post-order traversal of the tree. This ensures that MBRs are updated from the leaves up to the root.

- **Leaf Pages:** The routine iterates through tuples and invokes the visibility callback. Valid tuples are kept, while "dead" tuples are filtered out.
- **Internal Pages:** The routine recursively visits child nodes. If a child becomes entirely empty, its downlink in the parent is pruned.
- **MBR Re-aggregation:** After filtering, the page is initialized and rebuilt with the remaining tuples. The new page MBR is recomputed using the `tr_union` operator and returned to the parent to update the upper-level bounding boxes.

```

1 /* Initialize aggregate box as empty */
2 MemSet(&aggregate, 0, sizeof(aggregate));
3 aggregate.flags = TRTREE_FLAG_EMPTY;
4
5 /* Re-add surviving tuples and update the aggregate MBR */
6 for (i = 0; i < nitems; i++) {
7     rtree_add_tuple_copy(page, tuples[i]);
8     aggregate = (aggregate.flags & TRTREE_FLAG_EMPTY) ?
9                 boxes[i] : tr_union(&aggregate, &boxes[i]);
10 }
11 *page_box = aggregate; // Return updated MBR to parent

```

Listing 5: MBR Re-aggregation during Vacuum

Root Shrinking To prevent the tree from becoming unnecessarily deep after significant deletions, we implemented `rtree_vacuum_shrink_root`. If a root page is reduced to a single child, the root is "collapsed"—the child page's contents are moved to the root block, and the tree height is decremented. This optimization maintains logarithmic search performance even in volatile datasets.

Durability Vacuum operations utilize the `rtree_wal_vacuum` wrapper to log page modifications. During root shrinking, where multiple blocks (the metadata page and the root page) are modified simultaneously, we utilize `GenericXLogStart` to ensure the structural change is atomic and crash-recoverable.

6.3.9 WAL Logging and Durability (`temporal_rtree_wal.c`)

To ensure ACID compliance and crash recovery, the index utilizes PostgreSQL’s `GenericXLog` facility. The implementation provides specialized wrappers for atomic operations:

- **Insert/Vacuum:** Registers the modified buffer using `GENERIC_XLOG_FULL_IMAGE` to ensure consistency during tuple addition or reclamation.
- **Split Operations:** Employs a single WAL transaction to register both left and right buffers, guaranteeing that page-linking and MBR updates are atomic and durable.

6.3.10 Execution and Planning Hooks (`temporal_rtree_hooks.c`)

To integrate with the PostgreSQL kernel without altering core source code, the extension utilizes the Hook mechanism to intercept planner and executor processes. This enables real-time monitoring and optimizer influencing through three primary entry points:

- **Planner Hook (`trtree_planner_hook`):** Intercepts query trees to detect the presence of `temporalbox` functions and determines if a relation is eligible for R-tree indexing. It maintains global statistics on how often the index is considered by the optimizer.
- **Pathlist Hook (`trtree_set_rel_pathlist_hook`):** Dynamically biases the cost model. When enabled, it applies a `COST_BIAS_FACTOR` (e.g., 0.05) to `IndexPath` nodes associated with the `temporal_rtree`, making the access method significantly more attractive to the planner during cost-based optimization.
- **Executor Hook (`trtree_executor_start_hook`):** Monitors DML operations (INSERT, UPDATE, DELETE) targeting indexed relations. This hook facilitates logging and instrumentation of index write performance in a production environment.

The implementation also exposes SQL-accessible functions, such as `temporal_rtree_hook_stats()`, which allow administrators to query internal hit counters and evaluate index eligibility across the workload.

6.3.11 Cost Estimation (`temporal_rtree_cost.c`)

Cost estimation integrates with PostgreSQL’s planner to provide accurate selectivity and I/O cost estimates for temporal range predicates. The estimator models the probability that a random data MBR overlaps the query box using the fraction of the 2D space covered by the query, tuned with statistics gathered during `ANALYZE`.

7 Comparative Performance Analysis

The following tables summarize the results of our benchmarking across 100,000 generated temporal rows. Table 1 shows execution time across all DML operations, while Table 2 reports buffer usage and index storage footprint.

The `temporal_rtree` achieves a reasonable SELECT latency of 7.050 ms. This performance comes from leaf-level filtering over the two-dimensional `attr`-time representation, while internal MBR pruning is kept disabled in the correctness-safe version to avoid false negatives. The

Table 1: Execution Time Comparison (ms)

Index Config	SELECT	INSERT	UPDATE	DELETE
No Index	18.006	64.707	59.209	26.674
BRIN	19.119	66.362	60.191	24.651
B-Tree	19.185	96.703	68.757	25.048
GiST (Period)	2.608	283.464	173.724	0.065
HST-GiST	20.040	277.237	164.935	0.089
Hybrid Current-History	19.050	244.952	161.548	0.068
GiST (Attr Period)	6.322	353.458	261.617	24.442
Temporal R-Tree	7.050	641.181	470.869	24.115

higher INSERT and UPDATE costs are expected, since each write must maintain the custom two-dimensional index structure, update bounding boxes, and handle page-level changes. This behavior is consistent with spatially aware indexes such as GiST on composite attributes.

Table 2: Index Size and Read Efficiency

Index Config	Shared Hit	Shared Read	Total Index Size (kB)
No Index	59	0	2208
BRIN	82	0	2232
B-Tree	87	0	5128
GiST (Period)	81	0	8144
HST-GiST	81	0	3344
Hybrid Current-History	107	0	5712
GiST (Attr Period)	185	0	14304
Temporal R-Tree	185	0	25832

The `temporal_rtree` incurs a similar order of shared-buffer activity to GiST on the composite attribute-period representation. In the final version, internal pruning is conservatively disabled and filtering is performed at the leaf level, which explains why the SELECT latency remains behind mature GiST. However, the index still maintains a compact representation of the two-dimensional temporal boxes, and its storage footprint remains competitive with the GiST-based composite baseline. The higher write-side cost reflects the overhead of maintaining this custom R-tree structure, including tuple insertion, page updates, and WAL logging.

8 Conclusive Remarks and Future Work

The project successfully demonstrated that a custom 2D R-tree Access Method can provide superior read performance for composite temporal queries compared to generic GiST wrappers. Future work will focus on:

- Implementing R* tree split logic to further reduce overlap and improve insertion efficiency.
- Enhancing the delta-logging mechanism to minimize WAL overhead.
- Introducing page-level locking for concurrent workloads.
- Exploring adaptive split strategies based on workload patterns.

9 References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed., Chapters 14 & 24).
2. Guttman, A. (1984). “R-trees: A Dynamic Index Structure for Spatial Searching.” *SIGMOD*.
3. PostgreSQL Global Development Group. *PostgreSQL Documentation: Index Access Method Interface*.
4. project chat: <https://chatgpt.com/share/69f321fd-6cd8-83a4-b30f-a29acae72e73>
This research utilized Gemini for critical debugging and optimization. AI-assisted analysis of WAL logging and search traversal resolved “false negative” errors and reduced UPDATE latencies from 1000ms to 350ms. Additionally, Gemini was employed to synthesize the final L^AT_EX report and derive conclusions from experimental performance data. Ai also helped us figure out the possible things we can implement and test.

A Benchmark Queries

The following queries were used during benchmarking. Each was executed after a DISCARD ALL to clear shared-buffer caches, and timing was measured via EXPLAIN (ANALYZE, BUFFERS).

```
1 -- SELECT: point-in-time composite lookup
2 SELECT * FROM temporal_data
3 WHERE   attr_id = 42
4   AND   valid_period && tsrange('2024-01-01', '2024-06-01');
5
6 -- INSERT: single row with open-ended interval (current row)
7 INSERT INTO temporal_data (attr_id, valid_period, payload)
8 VALUES (9999, tsrange('2026-01-01', NULL), 'test_payload');
9
10 -- UPDATE: close the current row and open a new version
11 UPDATE temporal_data
12 SET    valid_period = tsrange(lower(valid_period), '2026-04-30')
13 WHERE  attr_id = 9999
14   AND  upper_inf(valid_period);
15
16 -- DELETE: remove all historical rows for an attribute
17 DELETE FROM temporal_data
18 WHERE  attr_id = 9999
19   AND  NOT upper_inf(valid_period);
```

Listing 6: Benchmark Queries

B Source File Reference

The table below summarises the role of each source file in the extension.

- `temporal_rtree.h` / `temporal_rtree_private.h`: Public and private header files defining RTreeTemporalBox, state structures, and normalization helpers.
- `temporal_rtree_handler.c`: Defines the `IndexAmRoutine` API entry point and registers all AM callbacks.
- `temporal_rtree_build.c`: Implements index build and bulk-loading with centroid-sorted bottom-up packing.

- `temporal_rtree_insert.c`: Handles single-tuple insertion with `choose_subtree` and upward MBR adjustment.
- `temporal_rtree_search.c`: Implements stack-based depth-first search with MBR predicate pruning.
- `temporal_rtree_split.c`: Implements the $O(N)$ Linear Split algorithm for page overflow.
- `temporal_rtree_page.c`: Manages page-level tuple slots and MBR recomputation.
- `temporal_rtree_vacuum.c`: Handles bulk dead-tuple removal and post-vacuum MBR consolidation.
- `temporal_rtree_wal.c`: Implements WAL logging with delta-range optimizations to minimize I/O.
- `temporal_rtree_hooks.c`: Implements the `planner_hook` and `ExecutorStart_hook` for non-intrusive monitoring.
- `temporal_rtree_cost.c`: Provides selectivity and I/O cost estimates for the PostgreSQL planner.